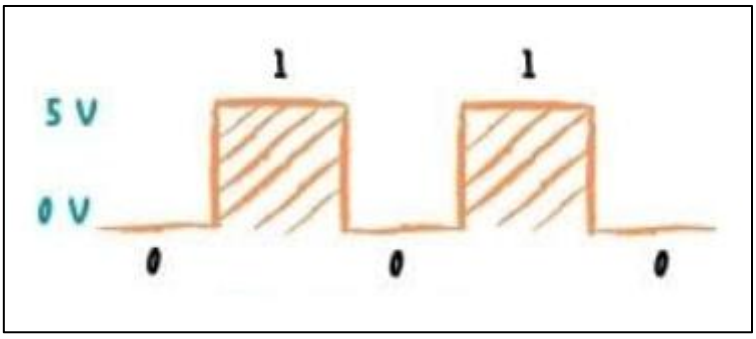
Lesson 10 RRC DC Brushed Motor AB Quadrature Encoder Principles and Source Code Analysis

1. Preface

DC gear motor is a high-torque, high-load motor available in 8V and 12V variants, equipped with an encoder. This chapter focuses on controlling the operation of the JGA27-310 DC gear motor (referred to as the 310 motor) using an STM32 main control board.

Principle of AB quadrature encoder:

The rotary quadrature encoder is a digital IC (integrated circuit), so it uses binary code. Square waves indicate digital signals in high (1) and low (0) states.



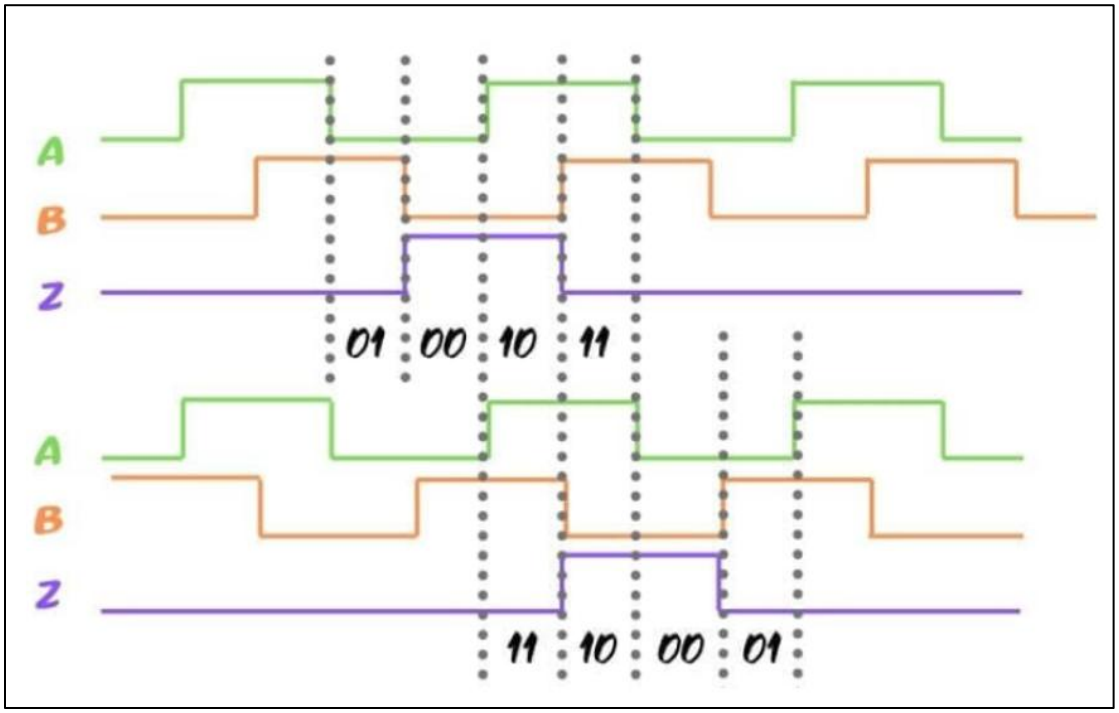
The operation of the encoder under 5V (using 5V as an example) power supply is illustrated in the above diagram, where the lines show low 0V (ground) and high 5V states.

Velocity determination: The number of pulses per revolution of the disk depends on the number of marks on its surface. This principle is the basic definition of encoder resolution. Therefore, two output signals represent the

number of pulses in a specific time period. Using these values, the control system can determine angular velocity, distance, and acceleration.

Direction determination:

Two pins are placed sequentially. Thus, when simultaneously viewing the two waveform lines, you will notice a slight offset. Output channels A and B are specifically positioned on the disk at a certain distance, causing them to have a 90-degree phase offset on the waveform. The cycle is also divided into 4 equal parts.



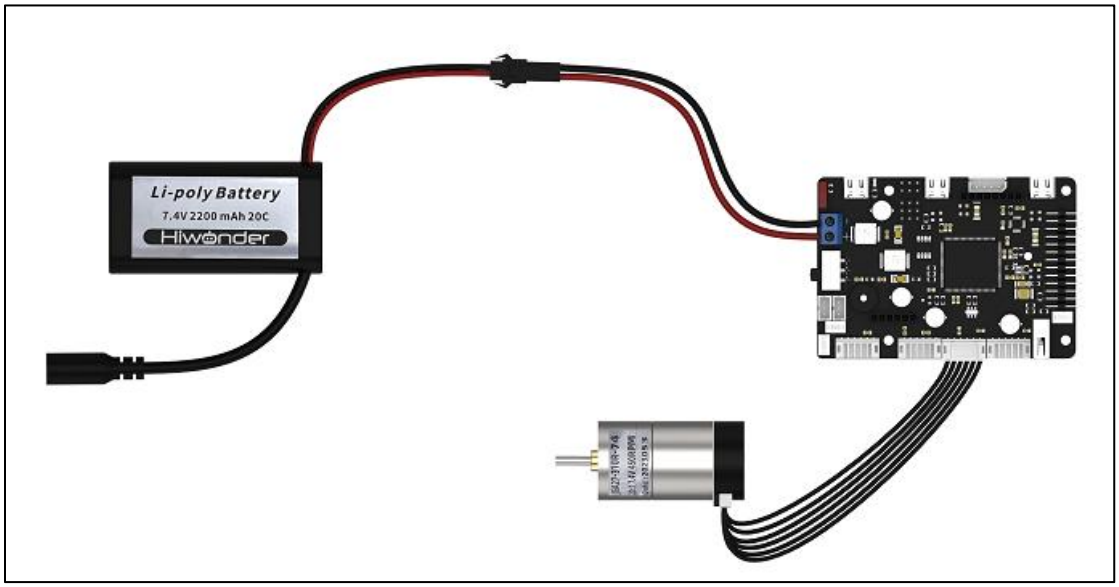
If the encoder moves forward (clockwise), line A is at the highest point (1), and channel B is at the lowest point (0). This results in combinations like 1 and 0, followed by 11, 00, and so on.

Starting from the same point in different directions (counterclockwise), you will see that this line will first be at the highest point (11), followed by combinations like 01, 00, and so on.

2. Hardware Connection

Required materials: 7.4V lithium battery pack, STM32 main control board, 310 motor.

The hardware connection diagram is as shown below:



Wiring method: Connect the 7.4V lithium battery and the DC power line of the STM32 main control board, then connect the 310 motor to the corresponding motor interface of the STM32 main control board according to the diagram.

3. Program Download

The example routine for this content is named "EncoderMotor_PID_Demo":

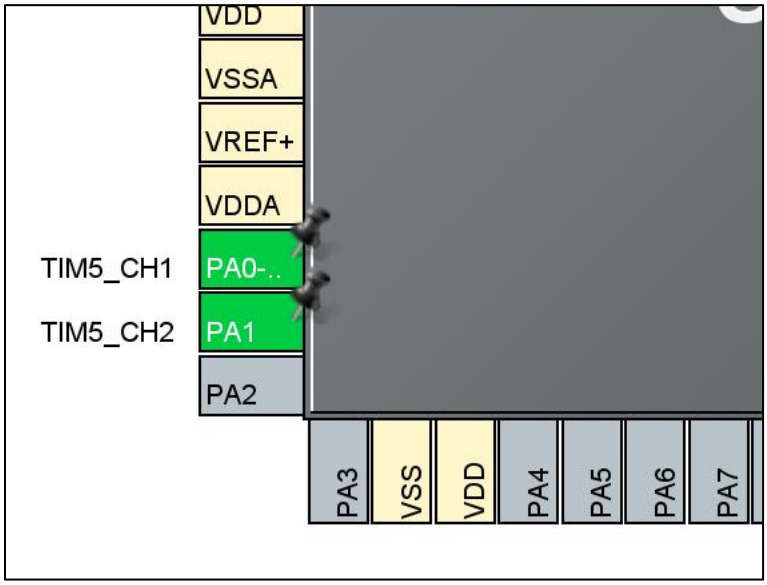
Specific instructions for program download can be found in "**2. STM32 Development Fundamentals / Lesson 3 Project Compilation and Download**" of the document. Simply replace the hex file mentioned in the document with the hex file generated from this example routine (EncoderMotor_PID_Demo.hex) for downloading.

4. Program Outcome

After powering on the STM32 control board, the motor rotates slowly for a period of time, then rotates quickly for another period of time, cycling repeatedly between these two operating states.

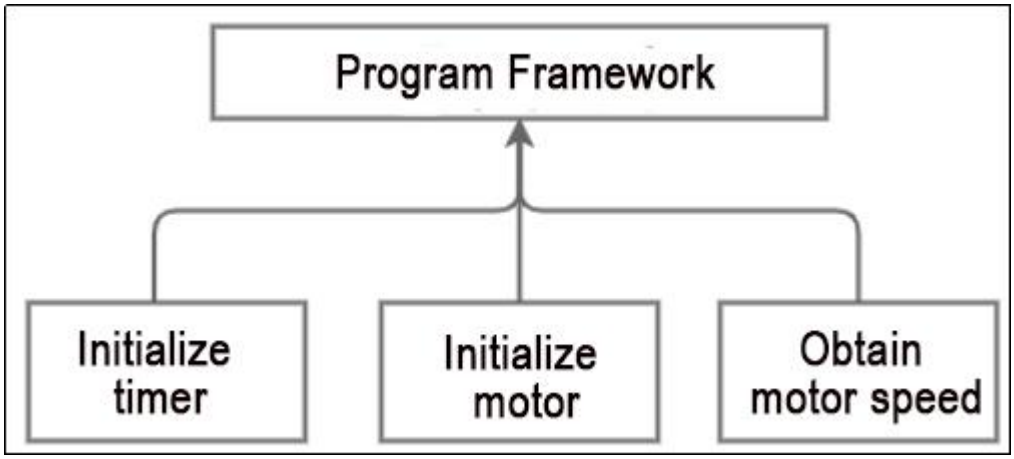
5. PIN Configuration

Using STM32CubeMX software to view pin information, during program execution, the external interrupt function will enable TIM5 timer loading. According to the diagram below, the corresponding chip pins are: PA0 and PA1.



6. Source Code Analysis

The program source code for this course is located in the "**Encoder_Demo**" folder. The program framework is designed based on the functionality as shown in the diagram below, with the main analysis conducted in the **main.c** file.



The program is divided into three parts: timer initialization, motor initialization, and motor speed acquisition.

By initializing the timer and motor, and acquiring the motor speed, the program verifies whether the AB phase control direction of the motor is correctly connected.

6.1 Initialize Timer

```
MX_TIM1_Init();
MX_TIM5_Init();
MX_TIM6_Init();
```

In the main function of the main.c file, three timers are initialized as shown in the diagram below: timers 1, 5, and 6 are configured with parameters. You can also refer to the tim.c file for details. These three timers have been configured through the interface in the STM32CubeMX software (it is recommended to use STM32CubeMX software for a more intuitive view).

Counter Settings	
Prescaler (PSC - 16 bits value)	839
Counter Mode	Up
Counter Period (AutoReload Register - 16 bit...	999
Internal Clock Division (CKD)	No Division
Repetition Counter (RCR - 8 bits value)	0
auto-reload preload	Enable

Timer 1

Counter Settings	
Prescaler (PSC - 16 bits value)	0
Counter Mode	Up
Counter Period (AutoReload Register - 32 bit...	60000
Internal Clock Division (CKD)	No Division
auto-reload preload	Enable

Timer 5

Counter Settings	
Prescaler (PSC - 16 bits value)	83
Counter Mode	Up
Counter Period (AutoReload Register - 16 bits...	10000
auto-reload preload	Disable

Timer 6

6.2 Motor Initialization

Before controlling the motor operation, it is necessary to initialize the relevant parameters of the motor in the main function located in the main.c file.

```
motor_init();
```

```
/**
 * @brief 电机初始化
 * @detials 初始化电机相关内存及定时器
 * @retval None.
 *
 */
void motor_init(void)
{
    encoder_motor_object_init(&motor1); /* 初始化结构体 */
    motor1.ticks_per_circle = MOTOR_DEFAULT_TICKS_PER_CIRCLE; /* 设置电机每转产生的脉冲总数， 用于计算转速转换为转每秒*/
    motor1.ticks_overflow = 60000;
    /* 电机控制 */
    __HAL_TIM_SET_COUNTER(&htim1, 0); /* 清除定时器计数 */
    __HAL_TIM_ENABLE(&htim1);         /* 启动PWM信号生成定时器 */
    __HAL_TIM_MOE_ENABLE(&htim1);     /* 开启PWM信号输出 */

    /* 编码器 */
    __HAL_TIM_SET_COUNTER(&htim5, 0);               /* 清除定时器计数 */
    __HAL_TIM_CLEAR_IT(&htim5, TIM_IT_UPDATE);      /* 清除更新中断标志 */
    __HAL_TIM_SET_AUTORELOAD(&htim5, motor1.ticks_overflow); /* 设置计数溢出值， 可以通过cubemx设置 */
    __HAL_TIM_ENABLE_IT(&htim5, TIM_IT_UPDATE);     /* 开启更新中断， 用于计数溢出计算 */
    __HAL_TIM_ENABLE(&htim5);                       /* 启动定时器 */
    HAL_TIM_Encoder_Start(&htim5, TIM_CHANNEL_ALL); /* 启动定时器编码器模式 */

    /* 定时器 */
    /* 需要每10ms计算一次编码器数值来获取速度 */
    __HAL_TIM_SET_PRESCALER(&htim6, 83);        /* TIM6连接到APB1, 时钟为84MHz, 84分频==1MHz */
    __HAL_TIM_SET_AUTORELOAD(&htim6, 10000);    /* 1MHz == 1us/cycle, 10000us == 10ms, 每 10ms产生一次更新中断 */
    __HAL_TIM_SET_COUNTER(&htim6, 0);           /* 清除计数 */
    __HAL_TIM_CLEAR_IT(&htim6, TIM_IT_UPDATE);  /* 清除更新中断 */
    __HAL_TIM_ENABLE_IT(&htim6, TIM_IT_UPDATE); /* 开启更新中断 */
    __HAL_TIM_ENABLE(&htim6);                   /* 启动定时器 */
}
```

From the above diagram, it can be observed that:

The initialized structure (encoder_motor_object_init):

```
encoder_motor_object_init(&motor1); /* 初始化结构体 */
```

The parameters of motor1's structure are shown in the diagram, and the specific parameter contents can be referred to in the annotations. In the entire program, the parameter to pay attention to is rps, which is used to calculate the motor speed obtained.

```
/**
 * @brief 编码器电机对象结构体
*/
struct EncoderMotorObject {
    int64_t counter;        /**< @brief 总计数值，64bit 认为不会溢出 */
    int64_t overflow_num;   /**< @brief 溢出计数 */
    int32_t ticks_overflow; /**< @brief 计数溢出值 */
    float tps;              /**< @brief ticks per second 计数器频率 */
    float rps;              /**< @brief revolutions per second 输出轴转速 转每秒 */

    int32_t ticks_per_circle; /**< @brief 电机输出轴旋转一圈产生的计数个数，根据电机实际情况填写 */

};
```

Using Timer 1 to generate PWM signals, as shown in the diagram below:

```
/* 电机控制 */
__HAL_TIM_SET_COUNTER(&htim1, 0); /* 清除定时器计数 */
__HAL_TIM_ENABLE(&htim1);         /* 启动PWM信号生成定时器 */
__HAL_TIM_MOE_ENABLE(&htim1);     /* 开启PWM信号输出 */
```

Using TIM5 to control the encoder of the encoder motor for acquisition and startup, as shown below:

```
/* 编码器 */
__HAL_TIM_SET_COUNTER(&htim5, 0);               /* 清除定时器计数 */
__HAL_TIM_CLEAR_IT(&htim5, TIM_IT_UPDATE);      /* 清除更新中断标志 */
__HAL_TIM_SET_AUTORELOAD(&htim5, motor1.ticks_overflow); /* 设置计数溢出值， 可以通过cubemx设置 */
__HAL_TIM_ENABLE_IT(&htim5, TIM_IT_UPDATE);     /* 开启更新中断， 用于计数溢出计算 */
__HAL_TIM_ENABLE(&htim5);                       /* 启动定时器 */
HAL_TIM_Encoder_Start(&htim5, TIM_CHANNEL_ALL); /* 启动定时器编码器模式 */
```

Using TIM6 to calculate the encoder value once to obtain the speed, as shown below:

```
/* 定时器 */
/* 需要每10ms计算一次编码器数值来获取速度 */
__HAL_TIM_SET_PRESCALER(&htim6, 83);         /* TIM6连接到APB1, 时钟为84MHz, 84分频==1MHz */
__HAL_TIM_SET_AUTORELOAD(&htim6, 10000);     /* 1MHz == 1us/cycle, 10000us == 10ms, 每 10ms产生一次更新中断 */
__HAL_TIM_SET_COUNTER(&htim6, 0);            /* 清除计数 */
__HAL_TIM_CLEAR_IT(&htim6, TIM_IT_UPDATE);   /* 清除更新中断 */
__HAL_TIM_ENABLE_IT(&htim6, TIM_IT_UPDATE);  /* 开启更新中断 */
__HAL_TIM_ENABLE(&htim6);                    /* 启动定时器 */
```

6.3 Speed Obtaining

Using a while loop to obtain the motor speed and print it via serial port:

```
while (1) {
    printf("counter:%lld, speed:%0.3fr/s\r\n", motor1.counter, motor1.rps);
    HAL_Delay(100);
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
}
```

From the above diagram, it can be seen that the motor speed can be obtained through the motor.rps parameter. By printing the current motor rotation speed via serial port, it is possible to verify the accuracy of the motor's AB phase data. To ensure that the motor speed is not lost due to being too fast, a 100-millisecond delay is implemented using the HAL_Delay function.

Note: The main focus of the source code analysis is on the key structure of the DC motor drive program. This helps users quickly understand the structure and content of the code. Users can refer to the code and corresponding comments for detailed learning and reference.
